Operating Systems Lab — Assignment 10

Indian Institute of Technology Patna

Name: Ranveer Gupta
Roll Number: 2401MC20

1. Objective

The objective of this assignment was to implement and test physical memory tracking, virtual-to-physical address translation, page-table inspection and protection, and page replacement algorithms in the xv6-riscv operating system.

2. Implementation and Results

Q1. Physical Memory Tracking

Modified `kernel/kalloc.c` to maintain counters for the total number of allocated and freed physical pages. The `memstats()` function reports total allocated pages, total freed pages, currently allocated pages, and available physical pages.

A new `memstats` system call was implemented and tested using `user/memtest.c`. The test allocated and subsequently released three pages using `sbrk()`.

Observed Results:

Stage	Allocated	Freed	Currently Allocated	Available
Initial	240	40	200	32535
After allocation	243	40	203	32532
After deallocation	243	43	200	32535

The results confirm that allocation and deallocation are correctly tracked, and that the available page count returns to its initial value after releasing the allocated memory.

Q2. Virtual-to-Physical Address Translation

Implemented `vtop()` in `kernel/vm.c` to locate a valid user-page-table entry and calculate the corresponding physical address while preserving the page offset.

A new `vtop` system call was added and tested using `user/vttest.c`. The test verified successful address translation, preservation of the page offset, and rejection of invalid or unmapped addresses.

Observed Results:

- Virtual address `0x4000` was translated to physical address `0x87F26000`.

- An address with an offset of 100 was translated to `0x87F26064`.
- An invalid or unmapped address was correctly rejected.

These results demonstrate that the translation preserves the offset within the memory page.

Q3. Page-Table Inspection and Protection

Implemented two functions in `kernel/vm.c`:

- `pageinfo()`: Displays the physical frame address and the Present (P), Writable (W), and User (U) permission flags of a mapped user page.
- `setreadonly()`: Removes write permission from a mapped user page by clearing the `PTE_W` bit.

The corresponding `pageinfo` and `setreadonly` system calls were tested using `user/pagetest.c`.

Observed Results:

- Before protection: `P=1, W=1, U=1`.
- After protection: `P=1, W=0, U=1`.
- A child process attempting to write to the read-only page triggered the expected store/AMO page fault.
- The parent process survived the child's expected page fault.

This verifies that write protection was successfully applied to the page.

Q4. Page Replacement Algorithms

Implemented three page replacement algorithms—FIFO (First-In First-Out), LRU (Least Recently Used), and Optimal—in `user/pagereplace.c`.

Reference String Used:

```
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1
```

Each algorithm was tested with 3, 4, and 5 frames. The program displayed the frame contents after each page reference and identified whether the reference resulted in a page hit or a page fault.

Comparison of Results:

Frames	Algorithm	Hits	Faults	Hit Ratio	Fault Ratio
3	FIFO	5	15	25%	75%
3	LRU	8	12	40%	60%

3	Optimal	11	9	55%	45%
4	FIFO	10	10	50%	50%
4	LRU	12	8	60%	40%
4	Optimal	12	8	60%	40%
5	FIFO	11	9	55%	45%
5	LRU	13	7	65%	35%
5	Optimal	13	7	65%	35%

Observation: Optimal produced the fewest page faults for all three tested frame counts. LRU also performed better than FIFO for the given reference string. With 4 and 5 frames, LRU and Optimal produced identical fault counts.

3. Conclusion

All four tasks were implemented and tested successfully in the xv6-riscv operating system. The experiments demonstrated physical memory accounting, virtual-to-physical address translation, page-table permission inspection and modification, and the working of FIFO, LRU, and Optimal page replacement algorithms.

The observed outputs were consistent with the expected behaviour of the implemented mechanisms.